

</>

The Anuj DECODED

</>

```

01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};

```



```

01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver

```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3876 · Construct Uniform Parity Array II

LeetCode · Weekly Contest 494 · Part II of 3875

Medium

Math

Greedy

Parity

1. What Type of Question Is This?

This is **Part II** of "Construct Uniform Parity Array" — same setup as before, but with one extra rule bolted onto the subtraction option: **$\text{nums1}[i] - \text{nums1}[j]$ must be ≥ 1** , i.e. you may only subtract a *smaller* value. In Part I any two elements could pair up freely, so the answer was provably always true. Here, that ordering restriction breaks the symmetry: the **smallest element in the whole array can never use the subtraction rule** (there's nothing smaller to subtract), so it is forced into the direct-assignment rule. That single forced choice pins down the entire target parity for the array, and the rest of the problem is about checking whether every other element can still reach that forced target. This turns a pure counting problem into a **greedy / "smallest element decides everything"** problem.

2. What Is Given?

- An array **nums1** of n **distinct** integers.
- You must build another array **nums2** of the same length n where every element is the same parity — **either all odd or all even**.
- For every index i you must pick **exactly one** of two construction rules:

$\text{nums2}[i] = \text{nums1}[i]$	Copy the value straight across, unchanged
$\text{nums2}[i] = \text{nums1}[i] - \text{nums1}[j]$	Subtract a <i>smaller</i> element (index $j \neq i$, and $\text{nums1}[i] - \text{nums1}[j] \geq 1$)

This is the **only** difference from Part I — but it's enough to make the problem Medium instead of Easy, and to make the answer depend on the actual input again.

3. What Do We Need to Find?

Return **true** if some valid choice of rule (per index, respecting the new "subtract only a smaller value" restriction) lets every element of **nums2** end up the same parity, and **false** otherwise.

Function Signature

■ C++

```
class Solution {
public:
    bool uniformArray(vector<int>& nums1) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  nums1 = [1,4,7]
Output: true
```

Set $\text{nums2}[0] = \text{nums1}[0] = 1$ (direct, odd). Set $\text{nums2}[1] = \text{nums1}[1] - \text{nums1}[0] = 4 - 1 = 3$ (valid: $1 < 4$, and 3 is odd). Set $\text{nums2}[2] = \text{nums1}[2] = 7$ (direct, odd). Result $\text{nums2} = [1, 3, 7]$, all odd $\rightarrow$ **true**.

Example 2 — the interesting failing case

```
Input:  nums1 = [2,3]
Output: false
```

2 is the smallest value in the array, so it has **no** smaller element to subtract — its only option is $\text{nums2}[0] = 2$ (even). That forces the whole array's target parity to be **even**. Now look at 3: direct gives 3 (odd, doesn't match); subtracting is only allowed from a smaller value, and the only smaller value is 2, giving $3 - 2 = 1$ (still odd — doesn't match either). There is no way to make 3's contribution even, so the whole construction fails $\rightarrow$ **false**.

Example 3

```
Input:  nums1 = [4,6]
Output: true
```

Direct-copy both: $\text{nums2} = [4, 6]$, both even $\rightarrow$ **true**. No subtraction needed — the array was already uniform parity to begin with.

Constraints

- $1 \leq n == \text{nums1.length} \leq 10^5$
- $1 \leq \text{nums1}[i] \leq 10^9$
- nums1 consists of distinct integers.

n can be as large as 100,000 here (versus 100 in Part I) — any $O(n^2)$ approach will time out, so this version genuinely needs an $O(n)$ (or $O(n \log n)$) solution.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Find the one element that has no freedom at all.** The global **minimum** of `nums1` can never satisfy $\text{nums1}[i] - \text{nums1}[j] \geq 1$ for any j (nothing in the array is smaller than it). So the minimum's `nums2` entry is **forced** to be a direct copy of itself.
- **Step 2 — That forces the entire target parity.** Since `nums2`'s minimum-index entry must literally equal `nums1[minimum]`, the array's uniform target parity **must equal $\text{parity}(\min(\text{nums1}))$** . Unlike Part I, you don't get to freely try both targets — only one target is even on the table.
- **Step 3 — Work out what "fixing" a mismatch really requires.** For any other element v whose own parity doesn't match the target: $\text{parity}(v - j) = \text{parity}(v) \text{ XOR } \text{parity}(j)$. To flip a mismatch into a match you need $\text{parity}(j) = 1$ — in other words, **the helper j must be an ODD number**, regardless of what the target parity actually is. Subtracting an odd number always flips parity; subtracting an even number never does.
- **Step 4 — Add back the ordering rule.** That helper j must also be **smaller** than v . So a mismatched element v is fixable exactly when there exists some odd-valued element strictly less than v .
- **Step 5 — Specialize by whether the minimum itself is odd or even.** If the minimum is **odd**, it is itself an odd value smaller than every other element in the array — so it can single-handedly fix every mismatched (even) element. Target achievable, always. If the minimum is **even**, it can't help fix anything (it's not odd). Then any odd element in the array is a mismatch (target is even), and the *smallest* odd element has no smaller odd element to rescue it — it's stuck. So this case only works if there are **no** odd elements at all.
- **Step 6 — Collapse to one line.** Answer = true if the minimum of `nums1` is odd, OR if every element of `nums1` is already even. Otherwise false.

Which Algorithm Is Best?

A single $O(n)$ linear scan tracking just two things — the minimum value and whether any odd value exists — answers the whole question. There's no sorting, no per-pair search, and no data structure needed. The brute-force and "better" tiers below exist to show the discovery path: first the literal $O(n^2)$ translation of the rules, then an $O(n)$ version that checks feasibility per target using a precomputed "smallest odd value", and finally the fully collapsed $O(n)$ closed-form that needs only one pass and two variables.

Approach 1 — Brute Force (Search Every Pair for a Witness)

Idea: The most literal translation of the problem statement: for each of the two possible targets, check every index against every other index, honoring the new " $\text{nums1}[i] - \text{nums1}[j] \geq 1$ " rule, and see if a valid witness j exists wherever the direct value doesn't already match.

Steps

- Try target parity = even (0) first.
- For each index i , check if $\text{nums1}[i]$ itself already has the target parity. If so, move on.
- Otherwise, loop over every other index j , skip it if $\text{nums1}[i] - \text{nums1}[j] < 1$ (rule violation), and check whether the resulting difference has the target parity.
- If no valid j exists for some index, target = even fails entirely.
- Repeat for target parity = odd (1).
- Return true if either target parity works for every index.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool feasible(vector<int>& nums1, int target) {
        int n = nums1.size();
        for (int i = 0; i < n; i++) {
            bool found = (((nums1[i] % 2) + 2) % 2 == target);
            if (!found) {
                for (int j = 0; j < n; j++) {
                    if (j == i) continue;
                    int diff = nums1[i] - nums1[j];
                    if (diff < 1) continue; // must satisfy the new "≥ 1" rule
                    if (diff % 2 == target) { found = true; break; }
                }
            }
            if (!found) return false; // this index has no valid choice
        }
        return true;
    }

    bool uniformArray(vector<int>& nums1) {
        return feasible(nums1, 0) || feasible(nums1, 1); // try all-even, then all-odd
    }
};

int main() {
    vector<int> nums1 = {1, 4, 7};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl; // Expected output: true
    return 0;
}
```

■ Python

```
class Solution:
    def uniformArray(self, nums1):
        return self.feasible(nums1, 0) or self.feasible(nums1, 1)

    def feasible(self, nums1, target):
```



```

n = len(nums1)
for i in range(n):
    found = (nums1[i] % 2 == target)
    if not found:
        for j in range(n):
            if j == i:
                continue
            diff = nums1[i] - nums1[j]
            if diff < 1:
                continue          # must satisfy the new "≥ 1" rule
            if diff % 2 == target:
                found = True
                break
        if not found:
            return False          # this index has no valid choice
return True

if __name__ == "__main__":
    nums1 = [1, 4, 7]
    print(Solution().uniformArray(nums1))    # Expected output: True

```

■ JavaScript

```

class Solution {
    uniformArray(nums1) {
        return this.feasible(nums1, 0) || this.feasible(nums1, 1);
    }

    feasible(nums1, target) {
        const n = nums1.length;
        for (let i = 0; i < n; i++) {
            let found = (((nums1[i] % 2) + 2) % 2) === target;
            if (!found) {
                for (let j = 0; j < n; j++) {
                    if (j === i) continue;
                    const diff = nums1[i] - nums1[j];
                    if (diff < 1) continue;          // must satisfy the new "≥ 1" rule
                    if (diff % 2 === target) { found = true; break; }
                }
            }
            if (!found) return false;                // this index has no valid choice
        }
        return true;
    }
}

const nums1 = [1, 4, 7];
console.log(new Solution().uniformArray(nums1)); // Expected output: true

```

Complexity: $O(n^2)$ per target (nested loop over all pairs), $O(n^2)$ overall with 2 targets. With n up to 10^5 , this is roughly 10^{10} operations in the worst case — far too slow and will **time out** on the real judge, even though it's fully correct.

Approach 2 — Better (Precompute the Smallest Odd Value, Check Each Target in O(n))

Idea: Realize that fixing a mismatched element never needs a full search — it only needs to know *whether any odd value smaller than it exists anywhere*. That single fact is captured by one number: the smallest odd value in the whole array. Precompute it once, then checking each of the two targets becomes a simple O(n) pass instead of a nested search.

Steps

- Make one pass over `nums1` to find **smallestOdd** — the minimum value among all odd elements (or "none" if there are no odd elements at all).
- For target parity = even, then again for target parity = odd: scan every element `x`.
- If `x` already has the target parity, it's fine (direct assignment).
- Otherwise, it's fixable only if `smallestOdd` exists **and** `smallestOdd < x` — i.e., there's an odd value strictly smaller than `x` available to flip it.
- If every element passes this check for a given target, return true immediately.
- If neither target passes, return false.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool uniformArray(vector<int>& nums1) {
        long long smallestOdd = LLONG_MAX;
        for (int x : nums1)
            if (x % 2 != 0) smallestOdd = min(smallestOdd, (long long)x);

        for (int target = 0; target <= 1; target++) {
            bool ok = true;
            for (int x : nums1) {
                if (x % 2 == target) continue; // direct works
                if (smallestOdd != LLONG_MAX && smallestOdd < x) continue; // can flip via smallestOdd
                ok = false;
                break;
            }
            if (ok) return true;
        }
        return false;
    }
};

int main() {
    vector<int> nums1 = {1, 4, 7};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl; // Expected output: true
    return 0;
}
```

■ Python

```
class Solution:
    def uniformArray(self, nums1):
        odds = [x for x in nums1 if x % 2 != 0]
        smallestOdd = min(odds) if odds else None
```

```

    for target in (0, 1):
        ok = True
        for x in nums1:
            if x % 2 == target:
                continue
            if smallestOdd is not None and smallestOdd < x:
                continue
            ok = False
            break
        if ok:
            return True
    return False

if __name__ == "__main__":
    nums1 = [1, 4, 7]
    print(Solution().uniformArray(nums1))    # Expected output: True

```

■ JavaScript

```

class Solution {
    uniformArray(nums1) {
        let smallestOdd = null;
        for (const x of nums1) {
            if (x % 2 !== 0 && (smallestOdd === null || x < smallestOdd)) smallestOdd = x;
        }

        for (const target of [0, 1]) {
            let ok = true;
            for (const x of nums1) {
                if (x % 2 === target) continue;
                if (smallestOdd !== null && smallestOdd < x) continue;
                ok = false;
                break;
            }
            if (ok) return true;
        }
        return false;
    }
}

const nums1 = [1, 4, 7];
console.log(new Solution().uniformArray(nums1));    // Expected output: true

```

Complexity: One $O(n)$ pass to find `smallestOdd`, then up to two more $O(n)$ passes (one per target) — overall $O(n)$ time, $O(1)$ extra space. This comfortably handles $n = 10^5$, unlike Approach 1.

Approach 3 — Optimal (One Linear Scan, Closed-Form Answer)

Idea: Skip trying both targets altogether — Step 1–2 of the workflow already prove the target is **forced** to equal $\text{parity}(\min(\text{nums1}))$, so there's nothing to "try". Just track the minimum value and whether any odd value exists in a single pass, then apply the one-line rule derived in Step 6: the answer is true exactly when the minimum is odd, or when there are no odd numbers in the array at all.

Steps

- Make a single pass over `nums1`, tracking the running minimum (`minVal`) and whether any element is odd (`hasOdd`).
- If `minVal` is odd: the minimum itself is an odd value smaller than every other element, so it can fix every mismatched (even) element by subtraction — always achievable. Return true.
- If `minVal` is even: the target is forced to even, and any odd element becomes a mismatch that cannot always be rescued (the smallest odd value has no smaller odd value to fix it). So this only works when there are no odd values at all, i.e. `!hasOdd`.
- Return `minIsOdd || !hasOdd` — no loops beyond the single initial scan.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool uniformArray(vector<int>& nums1) {
        int minVal = INT_MAX;
        bool hasOdd = false;
        for (int x : nums1) {
            minVal = min(minVal, x);
            if (x % 2 != 0) hasOdd = true;
        }
        bool minIsOdd = (minVal % 2 != 0);
        return minIsOdd || !hasOdd;    // forced-target closed form, see workflow Step 6
    }
};

int main() {
    vector<int> nums1 = {1, 4, 7};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl;    // Expected output: true
    return 0;
}
```

■ Python

```
class Solution:
    def uniformArray(self, nums1):
        minVal = min(nums1)
        hasOdd = any(x % 2 != 0 for x in nums1)
        minIsOdd = (minVal % 2 != 0)
        return minIsOdd or not hasOdd    # forced-target closed form, see workflow Step 6

if __name__ == "__main__":
    nums1 = [1, 4, 7]
    print(Solution().uniformArray(nums1))    # Expected output: True
```

```
class Solution {
  uniformArray(nums1) {
    let minVal = Infinity;
    let hasOdd = false;
    for (const x of nums1) {
      if (x < minVal) minVal = x;
      if (x % 2 !== 0) hasOdd = true;
    }
    const minIsOdd = (minVal % 2) !== 0;
    return minIsOdd || !hasOdd;    // forced-target closed form, see workflow Step 6
  }
}

const nums1 = [1, 4, 7];
console.log(new Solution().uniformArray(nums1));    // Expected output: true
```

Complexity: A single $O(n)$ pass with $O(1)$ extra space — the minimum possible complexity for a problem that must at least read every element once. Compared to Approach 2 this removes the need to compute `smallestOdd` separately and to loop over the array twice more for the two targets; it's the same asymptotic class but with far fewer passes and branches.

Side-by-Side Comparison

1. Brute Force	Search every (i, j) pair for a valid witness, for both targets	$O(n^2)$ — TLEs at $n=10^5$	$O(1)$
2. Better	Precompute smallest odd value, check both targets in one pass	$O(n)$	$O(1)$
3. Optimal	Target is forced by the minimum; one linear scan, close to single pass	$O(n)$	$O(1)$

Key Takeaways to Remember

- **Look for the element with no freedom.** Whenever a rule only lets you pick from "smaller" or "earlier" elements, the global minimum (or first element) often has no valid alternative at all — and that forces everything downstream.
- **Subtracting an odd number always flips parity; subtracting an even number never does.** This is the general fact underneath both Part I and Part II — only the extra ordering constraint in Part II changes which helper values are actually usable.
- Adding a seemingly small constraint ("j must be smaller") can turn an always-true Easy problem into a genuinely input-dependent Medium one — re-check whether previous assumptions (like "any target is reachable") still hold before reusing a solution.
- When n scales up ($100 \rightarrow 10^5$), always re-examine an $O(n^2)$ brute force — what was "fine" for Part I's constraints will time out here.
- All three tiers were validated against **Example 1** ($\text{nums1} = [1,4,7] \rightarrow \text{true}$) across C++, Python, and JavaScript, and checked against Example 2's false case ($\text{nums1} = [2,3]$) by hand in the workflow above.